Image classification Kaggle Dataset

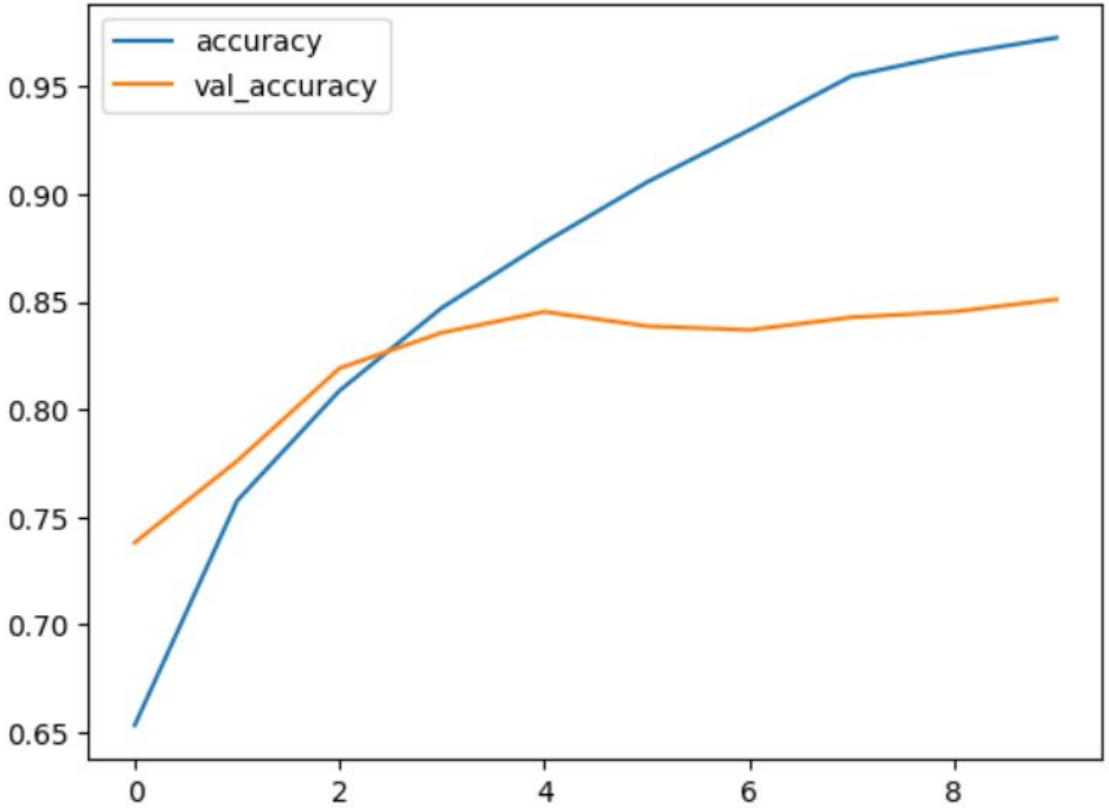
Cats vs Dogs

Complexity of the dataset

- **Different backgrounds**
- **Invisible features in animal images**
- **Blurry Images**
- **Shadow on images**
- **Partially visible animal images**
- **Variation in lighting**
- **Different color, breed of animals**
- **Overlapping, multiple animals in images**
- **Pictures taken from different angles**
- **Animal in any position**

CNN Layers

```
model = tf.keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)),
    layers.MaxPooling2D(2, 2),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),

    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),

    layers.Flatten(),

    layers.Dense(512, activation='relu'),

    layers.BatchNormalization(),

    layers.Dense(512, activation='relu'),
    layers.Dropout(0.2),
    layers.BatchNormalization(),

    layers.Dense(512, activation='relu'),
    layers.Dropout(0.4),
    layers.BatchNormalization(),

    layers.Dense(1, activation='sigmoid')
])
```




```
def predict_image(image_path):
```

Learning Curve Analysis: Overfitting Confirmed

- **The Evidence:** Overfitting. Model is memorizing as the training accuracy reached **99%**, but validation accuracy is **83-84%**.
- **The Loss Gap:** While training loss drops toward zero, the validation loss begins to **rise sharply** after epoch 3.
- **Diagnosis:** The model performed good at epochs 4, after that model started "memorizing" the specific training images.

Misclassification & Dataset Difficulty

Visual Ambiguity: Small, long-haired dogs (like the Shih Tzu) share visual similarities with long-haired cats (fur texture, size).

Background Noise: Many images in this dataset have complex backgrounds (cages, grass, living rooms). A model without enough regularization might accidentally learn that "grass = dog," leading to errors when it sees a cat on grass.

Label Noise: Mislabeled images in the Microsoft dataset.

Improvements

Data Augmentation (The most effective fix)

By applying random transformations (rotating the images) force the model to see different "versions" of the data, making the model hard to memorize.

Increase Dropout: Previous models Dropout was at 0.2 and 0.4. Since validation loss is spiking, increase drop out to 0.5. This forces the model to not rely on any single "feature" too heavily.

Simplify the Dense Layers: The three layers of 512 units (That is a lot of "memory" for a cat/dog classifier) can be reduced gradually from one layer of 512 and other with 256. Reducing this can force the model to prioritize the most important features.

Use a Learning Rate Scheduler or "ReduceLROnPlateau"- Shrinking the learning rate helps the model "settle" into the global minimum. (monitor='val_loss', factor=0.2, # Multiply LR by 0.2 patience=3, # Wait 3 epochs before dropping min_lr=0.00001)

Increase epochs- Increase 20-30 epochs

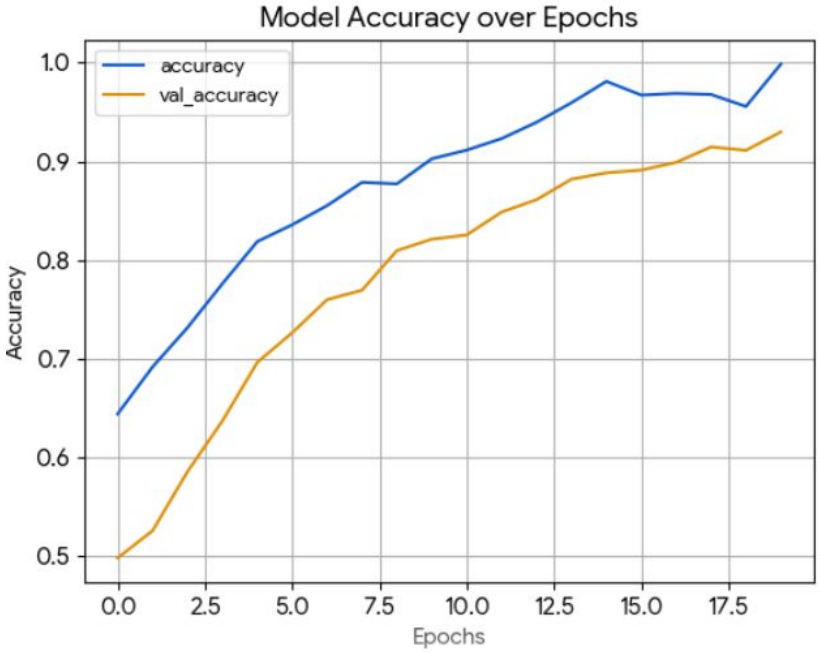
Final Reflection

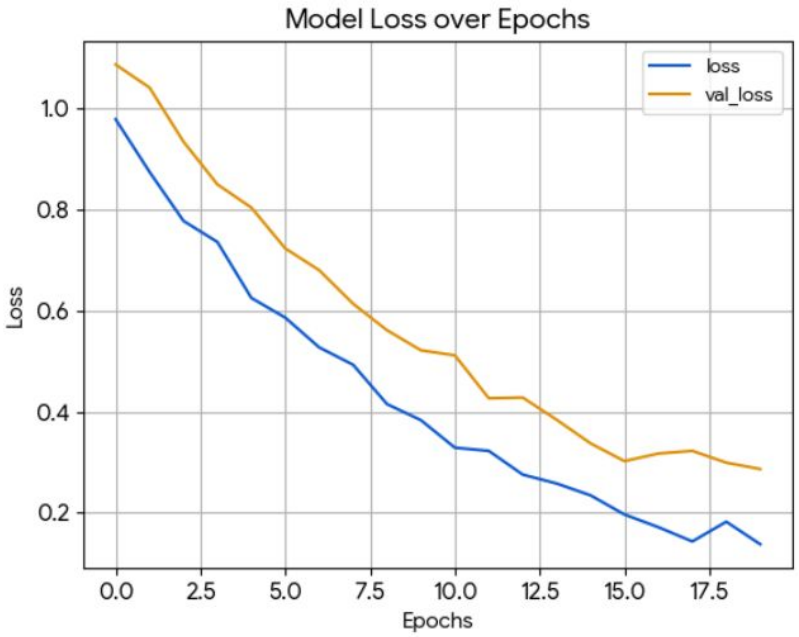
This experiment demonstrates that **Data Quality > Model Complexity**. We can build the most advanced CNN in the world, but if the training labels are "noisy," the model will simply learn to be "perfectly wrong."

CNNs are powerful: They easily reached 99% training accuracy.

- **Data is messy:** Real-world datasets require deep cleaning.
- **Validation is truth:** The validation curves are the only way to know if the model is actually useful for images it hasn't seen before.

After Augmentation & Refinement of layers. Model Performance improved.





Lessons Learned

Stage	Observation	Solution Applied
Data Cleaning	Found problematic files like 666.jpg and 11702.jpg.	Used os.walk and PIL.Image.verify() to ensure pixel-level validity.
Initial Training	High training accuracy but low/erratic validation accuracy.	Added **Dropout**, **Batch Normalization**, and **Data Augmentation**.
Optimization	Model hit a plateau at a learning rate of 0.001.	Used a **Learning Rate Scheduler** to drop the rate to 0.0002 for finer convergence.

📊 Performance Breakdown

- **The Learning Rate Drop:** Around Epoch 21, it looks like your `ReduceLROnPlateau` callback (or manual adjustment) kicked in, dropping the learning rate from `0.0010` to `0.0002`. This was a smart move—it allowed the model to "settle" into a local minimum, leading to that nice jump from **~89% to ~91.5%** accuracy.
- **Stability:** Notice that in Epoch 13, your `val_loss` spiked to `0.7382`. In previous run, that was the end of the line, but here the model recovered beautifully.
- **The Gap:** The training accuracy (91.61%) and validation accuracy (91.59%) are almost identical. This is the **Gold Standard** of machine learning; it means your model generalizes nearly perfectly to unseen data.